

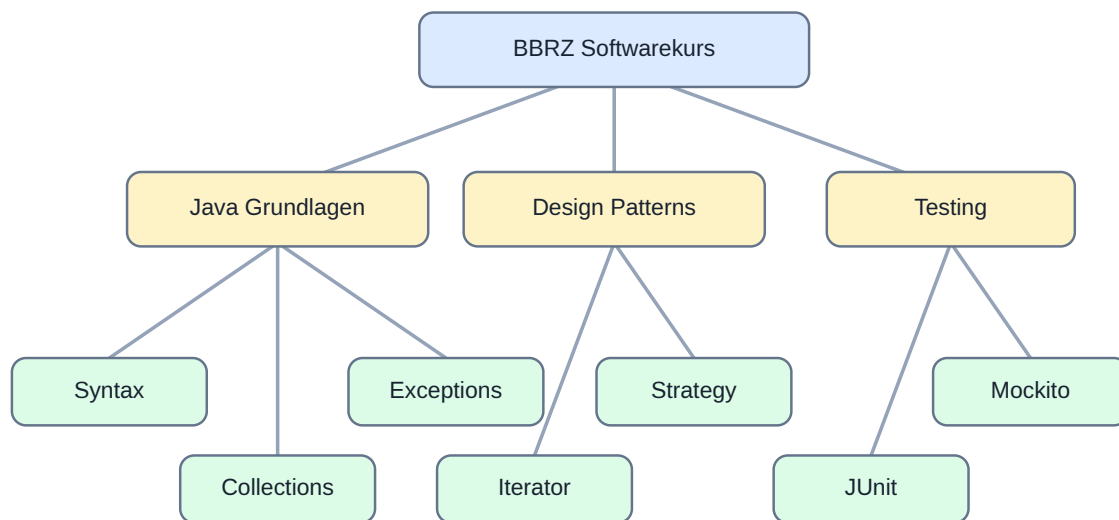
Iterator Pattern in Java 21

Erklärungen, Baumstruktur, Traversierungsschritte und Java-API-Beispiele

Das Iterator Pattern trennt die Frage „**Wie laufe ich durch die Daten?**“ von der Frage „**Wie sind die Daten gespeichert?**“. Der Client arbeitet nur mit `hasNext()` und `next()`. Die interne Struktur kann eine Liste, Queue, ein Stack, ein Baum oder etwas Komplexeres sein.

1. Beispielstruktur: Kursbaum

Alle eigenen Iteratoren arbeiten auf derselben Struktur. Nur die Traversierungslogik unterscheidet sich.



■ Wurzel ■ Container-Modul ■ Leaf/Lektion ohne Kinder

Iterator-Vertrag

```
while (iterator.hasNext()) {
    CourseModule module = iterator.next();
}
```

Der Client sieht nur den Iterator. Er kennt weder Baumstruktur noch Stack oder Queue.

Sammlung als Iterator-Fabrik

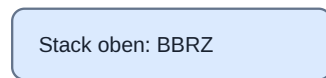
```
catalog.depthFirstIterator();
catalog.breadthFirstIterator();
catalog.lessonIterator();
```

Dieselbe Sammlung kann verschiedene Iteratoren liefern.

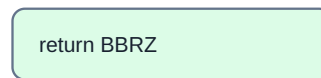
2. Depth-first Iterator: Stack

`DepthFirstModuleIterator` nutzt einen Stack. Ein Stack arbeitet nach **Last in, first out**. Die Kinder werden rückwärts auf den Stack gelegt, damit links zuerst verarbeitet wird.

1. Start



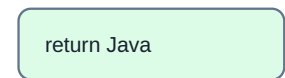
2. BBRZ poppen



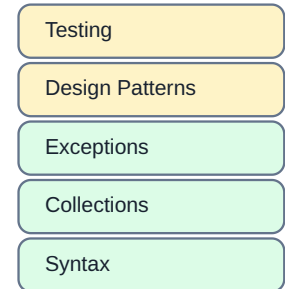
Kinder rückwärts pushen:



3. Java poppen



Kinder rückwärts pushen:



Ausgabe-Reihenfolge:

BBRZ, Java Grundlagen, Syntax, Collections, Exceptions, Design Patterns, Iterator, Strategy, Testing, JUnit, Mockito

```
CourseModule current = stack.pop();
List<CourseModule> children = current.children();
for (int i = children.size() - 1; i >= 0; i--) {
    stack.push(children.get(i));
}
return current;
```

3. Breadth-first Iterator: Queue

`BreadthFirstModuleIterator` nutzt eine Queue. Eine Queue arbeitet nach **First in, first out**. Dadurch werden zuerst alle Module einer Ebene verarbeitet.

1. Start

Queue vorne: BBRZ



2. BBRZ entfernen

return BBRZ

Kinder hinten anhängen:

Java

Patterns

Testing

vorne

hinten



3. Java entfernen

return Java

Patterns

Testing

Syntax

Collections

Exceptions

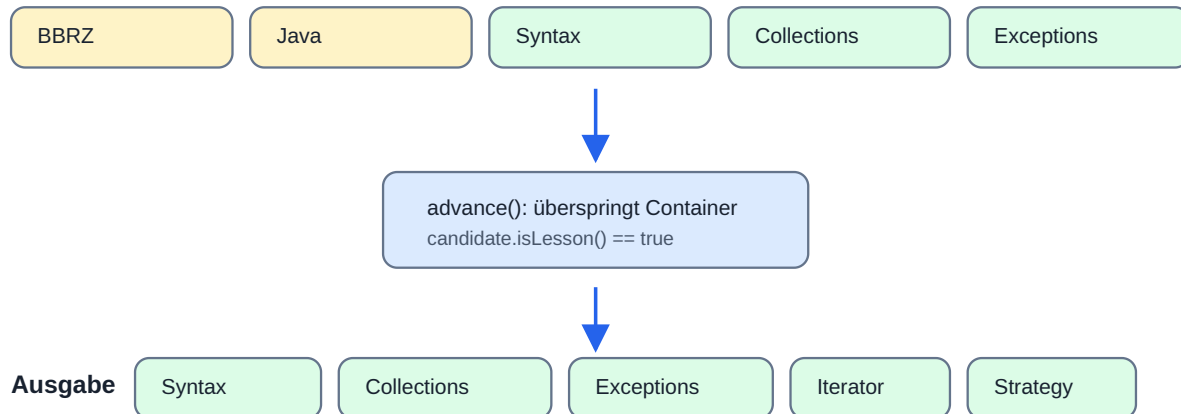
Ausgabe: BBRZ, Java Grundlagen, Design Patterns, Testing, Syntax, Collections, Exceptions, Iterator, Strategy, JUnit, Mockito

```
CourseModule current = queue.remove();  
queue.addAll(current.children());  
return current;
```

4. LeafLessonIterator: Iterator als Filter

`LeafLessonIterator` durchläuft intern alle Module, gibt aber nur Blätter zurück. Ein Blatt ist hier eine Lektion ohne Kinder. Der Iterator nutzt dafür einen anderen Iterator als Quelle.

Interner Iterator liefert alle Module



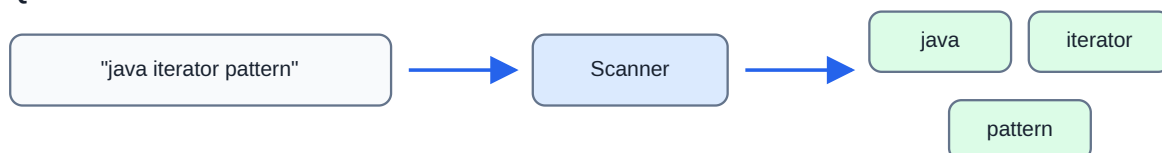
```
private void advance() {
    nextLesson = null;
    while (modules.hasNext() && nextLesson == null) {
        CourseModule candidate = modules.next();
        if (candidate.isLesson()) {
            nextLesson = candidate;
        }
    }
}
```

Der Basis-Iterator kann Depth-first oder Breadth-first sein. Der Filter bleibt gleich: Nur `CourseModule` -Objekte ohne Kinder werden ausgegeben.

5. Scanner: Iterator über Tokens

`Scanner` ist kein klassischer Collection-Iterator, verwendet aber dieselbe Grundidee: `hasNext()` fragt, ob noch ein Token vorhanden ist; `next()` liefert es.

Quelle



```
while (scanner.hasNext()) {
    tokens.add(scanner.next());
}
```

6. Splitterator: Durchlauf aufteilen

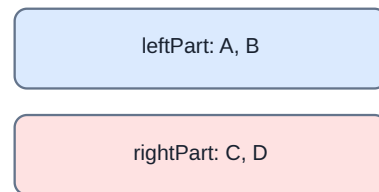
`Splitterator` steht für **split + iterator**. Er kann Elemente durchlaufen und den noch offenen Durchlaufbereich mit `trySplit()` aufteilen. Die Liste selbst wird dabei nicht verändert.

Liste bleibt unverändert



`trySplit()` →

Durchlaufbereiche



Ergebnis im Beispiel



```
Splitterator<String> rightPart = Arrays.asList("A", "B", "C", "D").splitterator();  
Splitterator<String> leftPart = rightPart.trySplit();
```

Wichtig: `trySplit()` garantiert nicht allgemein eine perfekte Halbierung. Bei geordneten Listen ist die zurückgegebene Teilmenge typischerweise der vordere Bereich.

7. Warum Java Iteratoren stark nutzt

Vorteile

- Die interne Datenstruktur bleibt verborgen.
- Der Client-Code bleibt einheitlich.
- Traversierungslogik ist austauschbar.
- Mehrere Iteratoren können unabhängig über dieselbe Sammlung laufen.
- Filter-Iteratoren können bestehende Iteratoren wiederverwenden.

Java-Beispiele

- `Collection.iterator()` für Listen, Sets und Queues.
- `Iterable` als Basis für die Enhanced-for-Schleife.
- `ListIterator` für Vorwärts- und Rückwärtsbewegung.
- `Scanner` für Token-für-Token-Verarbeitung.
- `Splitterator` als Basis für Streams und Aufteilung.

Gemeinsame Idee

